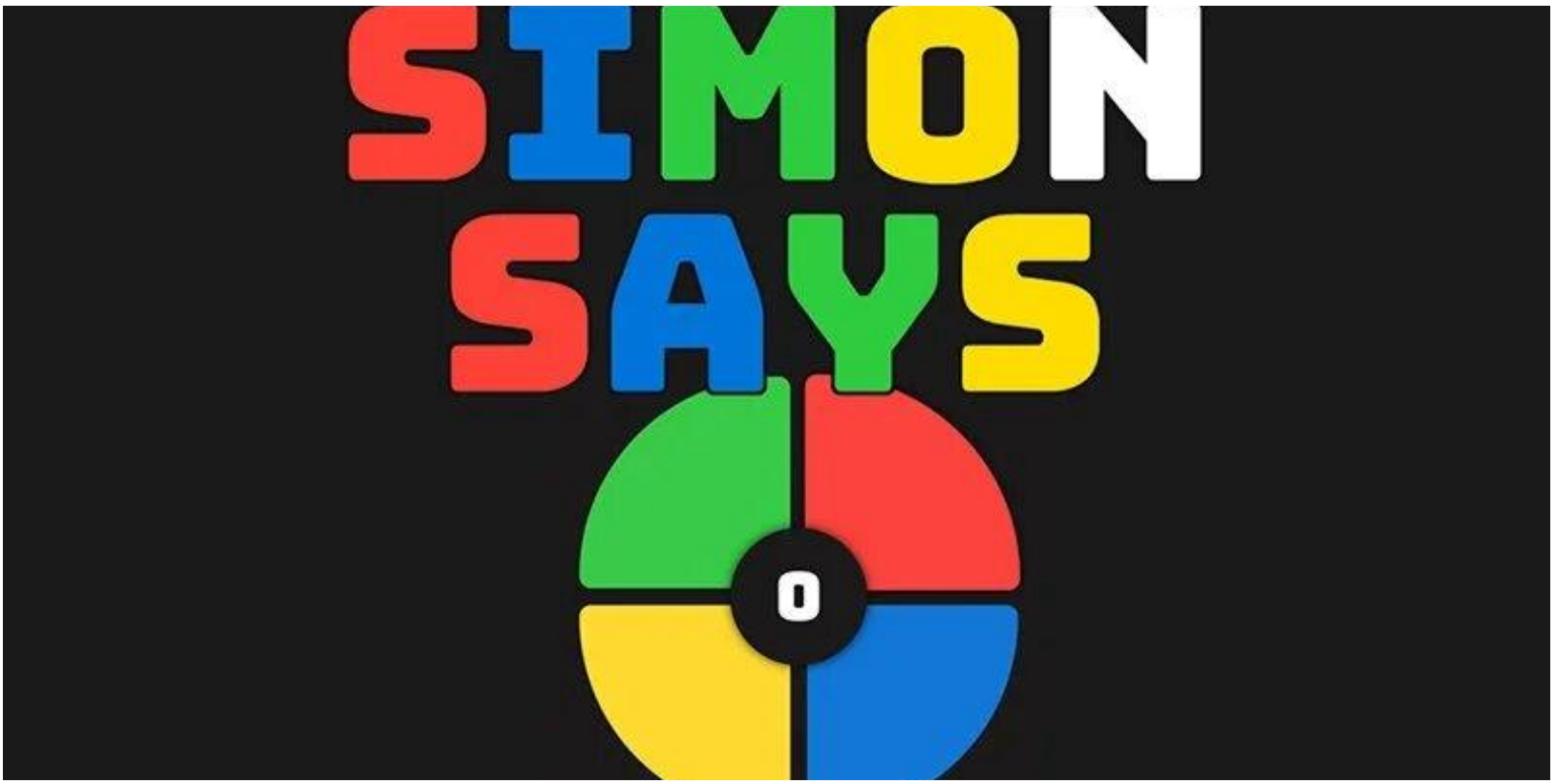
Simon Says Game

Programación Orientada a Objetos

05 de Mayo del 2023

Índice:

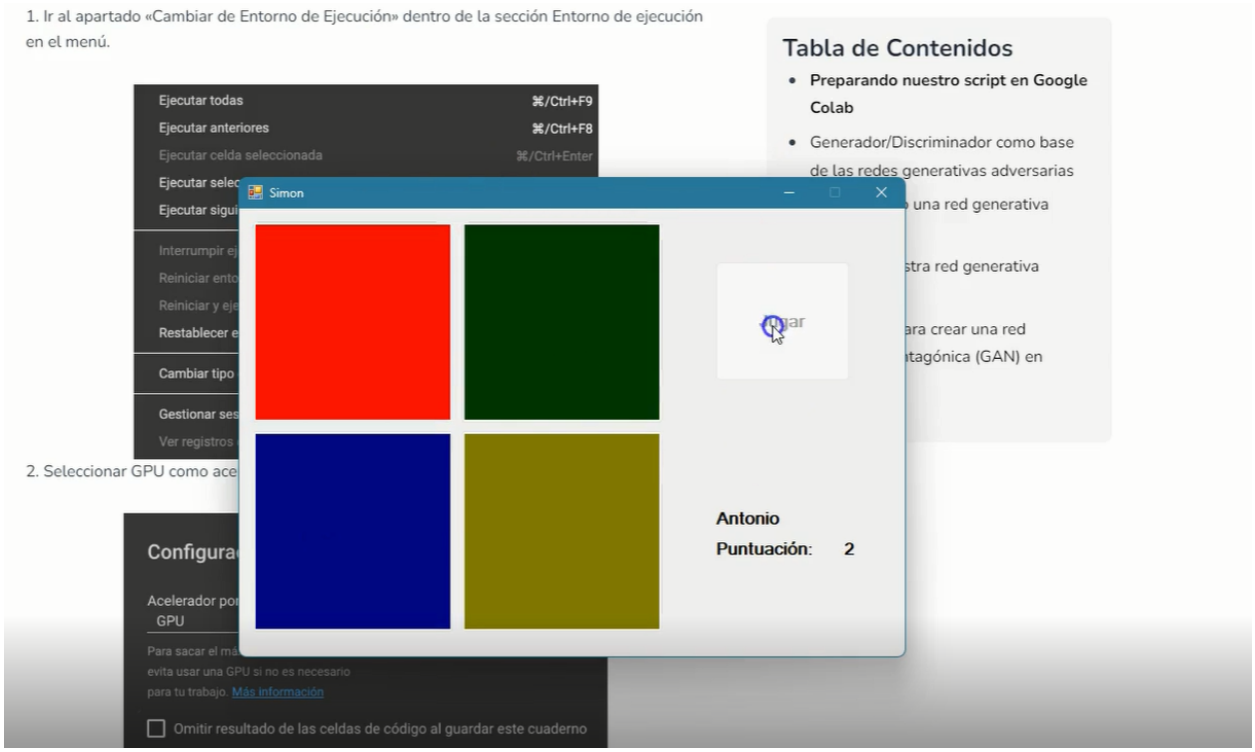
Introducción **2**
Diagrama de clases **3**
Ficheros **5**
Inconvenientes **6**
Conclusiones **7**



- Trabajo realizado por: Sandra Jiménez Ramiro

Introducción

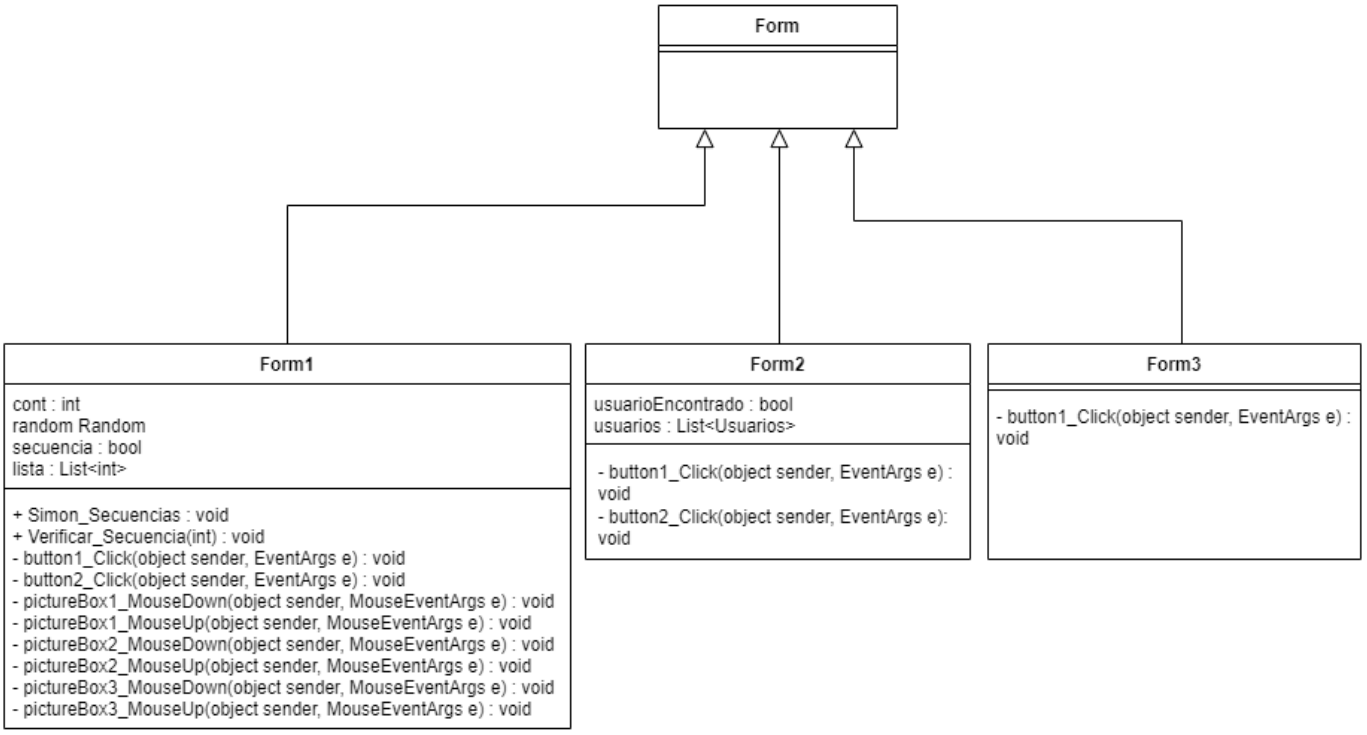
El juego de Simon Says es un juego clásico de memoria en el que los jugadores deben repetir una secuencia cada vez más larga de colores que se les presenta. He desarrollado esta práctica de C# usando Windows Form, en el cual he tratado de duplicar la práctica que se nos mostró en el vídeo de funcionamiento de la práctica.



Para el desarrollo de esta aplicación, he hecho uso de la manipulación de eventos, diseño de interfaces gráficas tanto para el propio juego como para las ventanas del login, uso de listas y manipulación de ficheros.

“

Diagrama de clases



Form: Es una clase predefinida en el espacio de nombres ‘System.Windows.Forms’, que se encuentra en la biblioteca de clases .NET Framework, esta proporciona una base para crear formularios que a su vez permiten la creación de interfaces gráficas de usuario en Windows Forms.

Form1: Tiene como variables un contador inicializado en 0, random de la clase Random que genera números aleatorios, un bool secuencia sentenciado en *false* y una lista de int’s.

La función Simon_Secuencias() que es la que se encarga de reproducir la secuencia de colores, secuencia se establece en *true,* lo que indica que se está reproduciendo una secuencia. Añado un tiempo en el que se detiene la ejecución durante unos milisegundos.

Creo un foreach que recorre la lista de colores, y dentro del bucle implemento un switch que muestra la imagen iluminada del color correspondiente, las cuales se encuentran en la carpeta llamada 'Resources'. Una vez que se han mostrado todas las imágenes de la secuencia, se establece el valor de secuencia en *false,* lo que indica que la reproducción de la secuencia ha terminado.

La función Verificar_Secuencia(int btn_presionado) sirve para verificar si el jugador ha seguido la secuencia correctamente. Primero verifica que no se esté mostrando una secuencia actualmente (secuencia = *false*) y que todavía hay elementos en la lista de secuencias. Si la variable "cont" es igual al nº de elementos en la lista, esto quiere decir que el jugador ha completado la secuencia correctamente, se muestra el mensaje, se reinicia el contador a 0 y se vacía la lista de secuencias. Si el botón presionado es mayor o igual que el elemento actual de la lista de secuencias, el jugador ha ingresado correctamente la secuencia y se incrementa el contador, por lo contrario, si el jugador pulsa un botón incorrecto, salta un mensaje de fallo.

Button1_Click() se usa para iniciar un nuevo juego y Button2_Click() para salir de la aplicación.

Los eventos "MouseDown" y "MouseUp" se utilizan para cambiar la imagen del botón seleccionado por otra imagen con un color más brillante para que se sepa que ese botón ha sido seleccionado.

Finalmente el evento pb1_Click, se usa para manejar los eventos de clic de los cuadrados de colores y se usa la función ya mencionada "Verificar_Secuencia()" para verificar si el botón presionado corresponde al siguiente elemento en la lista.

Form2: Este formulario se usa para validar las credenciales del usuario y permitir el acceso al juego.

Declaro una variable booleana "usuarioEncontrado" que indica si se ha encontrado el usuario en la lista de usuarios o no.

Declaro una lista de objetos “Usuarios” que se utiliza para almacenar los nombres de usuario y contraseñas.

El método button1_Click se ejecuta cuando el usuario hace clic en el botón “Iniciar sesión”, primero verifica que no hay campos vacíos en los cuadros de texto usando la función “string.IsNullOrWhiteSpace()”. Si alguno de ellos está vacío muestra un mensaje de error, de lo contrario, si están llenos se establecen las variables “usuario” y “contraseña” a los cuadros de texto correspondientes.

El método button2_Click se ejecuta cuando el usuario hace clic en el botón “Nuevo usuario” y se muestra un formulario (Form3) para permitir al usuario crear una nueva cuenta.

Form3: Este es un formulario que permite a los usuarios registrarse con un nuevo usuario. Si los campos se rellenan correctamente, se carga la lista de usuarios existente del fichero “archivo.txt” y se agrega un nuevo usuario a la lista. Luego, se escribe la lista completa de usuarios en el archivo. Después de guardarlo, se vuelve a abrir el Form2.

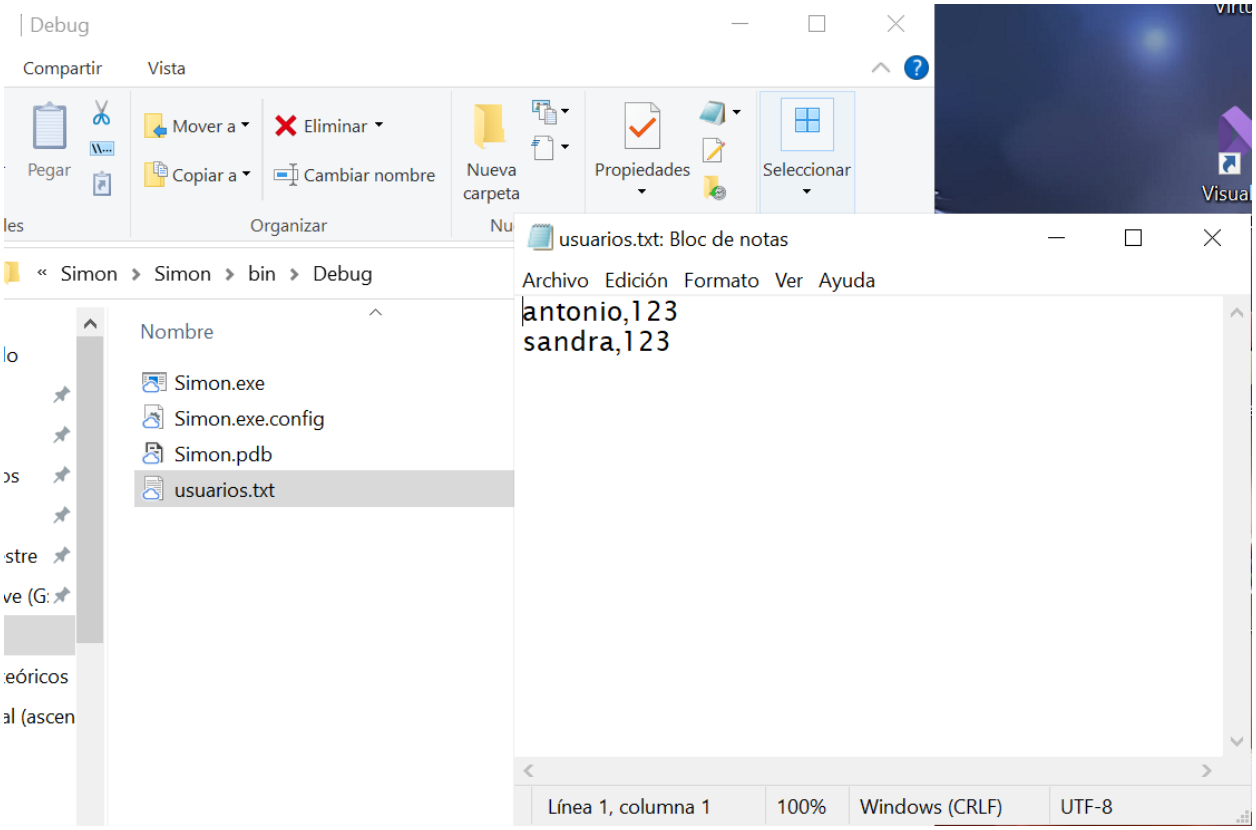
”

Ficheros

- - - - X

En cuanto a los ficheros, he usado en Form2, en la función “button1_Click” utilizo un archivo de texto “usuarios.txt” y lee el contenido con StreamReader. El contenido se divide en líneas, y cada línea se divide en dos partes, el nombre de usuario y la contraseña, ambos separados por una coma. Si el usuario y contraseña introducidos coincide con alguno de los que están guardados en el archivo, el booleano se establece en true y a continuación se muestra el juego (Form1).

Finalmente, el archivo “usuarios.txt” se utiliza para almacenar y leer la información de los usuarios. Cuando un usuario nuevo se registra (desde Form3), se lee el archivo existente, si existe, y se agrega el nuevo usuario a la lista de usuarios. Luego, se escribe la lista completa de usuarios en el archivo.



Inconvenientes

- - - - - - - - X

A la hora de realizar esta práctica he tenido algunos inconvenientes, por ejemplo, a la hora de crear el juego me saltaban excepciones como “índice fuera de la matriz” , lo he solucionado de una manera muy sencilla ya que era un despiste que no había visto, lo solucioné cambiando el nombre de los “pictureBox” y añadiendo una “_” para que de esta forma me funcionase el .Split(‘_’).

También tuve muchos problemas ya con el propio Visual Studio, ya que por alguna razón no se me modificaban los cambios realizados en la práctica y me he visto obligada a reiniciar en muchas ocasiones el pc.

Conclusiones

- - - - - - X

Conclusiones a las que he llegado haciendo la práctica:

1) Conocimiento adquirido: Durante todo este tiempo y no sólo con esta práctica, he aprendido muchas cosas de C#, he aprendido a usar ciertas funciones/métodos, he aprendido a aplicar contenidos vistos en la asignatura: herencia, composición, polimorfismo, gestión de errores, manipulación de ficheros, creación de formularios, etc.

2) Práctica y constancia: Esta práctica me ha permitido mejorar mis habilidades en programación y me ha enseñado a ser constante con las responsabilidades que tengo.

3) Creatividad: Aunque a la hora de realizar los formularios y el juego me haya inspirado en el vídeo que nos mostraron los profesores sobre el funcionamiento de la práctica, he disfrutado de poder hacer la interfaz yo misma y añadir las imágenes que más me convencían.